Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 4

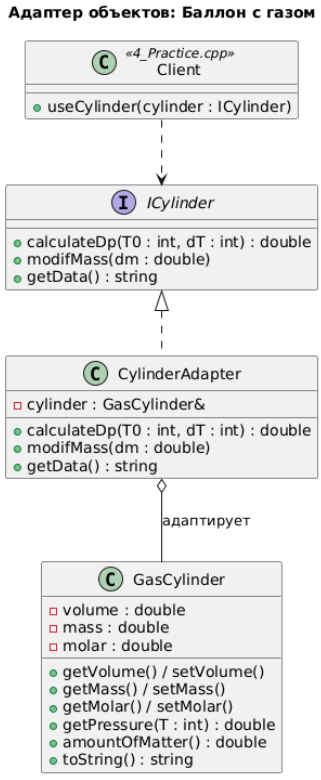
Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

1 объектов + CalculateDp(T0 : int, dT : int): double – определить изменение давления при заданной начальной температуре T0 и изменении темпе- ратуры dT; + ModifMass(dm: double):void – изменить массу газа в баллоне на величину dm; + GetData() : string – воз- вращает строку с данными об объекте

2 Расчёт ежедневной нормы потребления килокалорий. Классы (Тип телосложения): Астеник, Нормостеник, Гиперстеник. Параметры: Рост, Вес, Возраст, Пол Группа физической активности (низкая, средняя и высокая активность)

Диаграмма классов



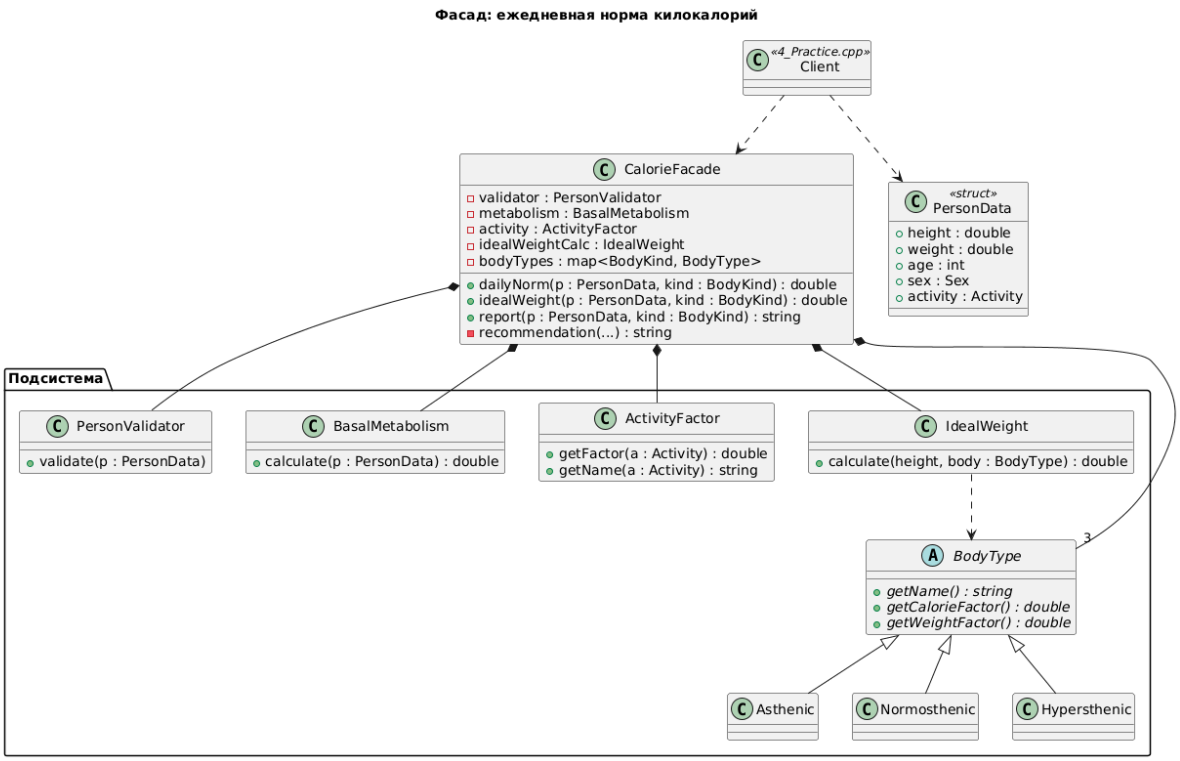
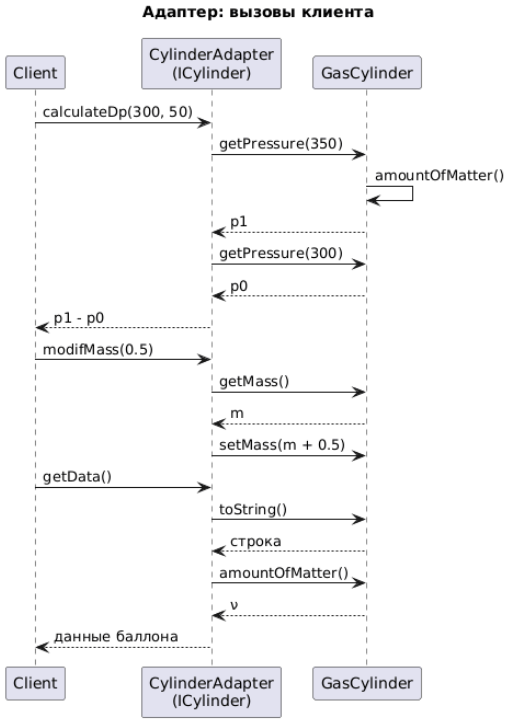
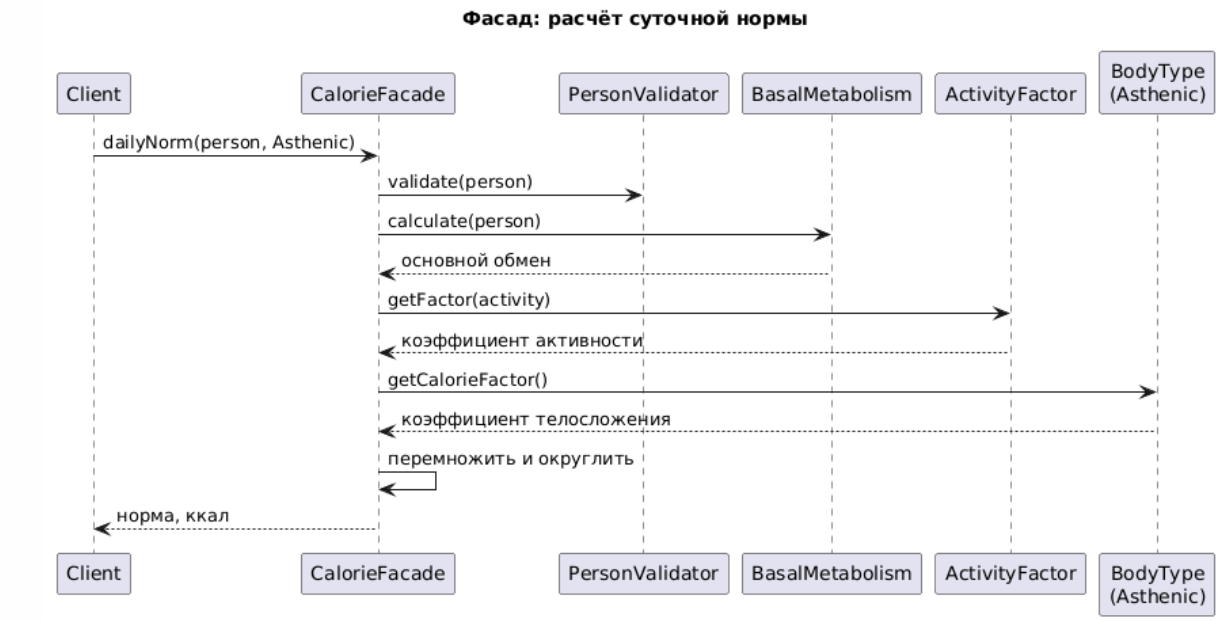


Диаграмма последовательности





Исходный код программы

4_CalorieFacade.h

```
#pragma once
#include <string>
#include <sstream>
#include <stdexcept>
#include <memory>
#include <map>
#include <cmath>

namespace Calories {

// ====================== Входные данные ======================

enum class Sex { Male, Female };
enum class Activity { Low, Medium, High };
enum class BodyKind { Asthenic, Normosthenic, Hypersthenic };

struct PersonData {
   double height;     // рост, см
   double weight;     // вес, кг
   int age;           // возраст, лет
   Sex sex;
   Activity activity;  // группа физической активности
};

// ====================== Классы подсистемы ======================
```

```
// Тип телосложения. Влияет на норму калорий и на идеальный вес.
class BodyType {
public:
   virtual ~BodyType() {}
   virtual std::string getName() const = 0;
   virtual double getCalorieFactor() const = 0;
   virtual double getWeightFactor() const = 0;
};

// Астеник: быстрый обмен веществ, легкий костяк
class Asthenic : public BodyType {
public:
   std::string getName() const override { return "астеник"; }
   double getCalorieFactor() const override { return 1.1; }
   double getWeightFactor() const override { return 0.9; }
};

class Normosthenic : public BodyType {
public:
   std::string getName() const override { return "нормостеник"; }
   double getCalorieFactor() const override { return 1.0; }
   double getWeightFactor() const override { return 1.0; }
};

// Гиперстеник: медленный обмен веществ, широкий костяк
class Hypersthenic : public BodyType {
public:
   std::string getName() const override { return "гиперстеник"; }
   double getCalorieFactor() const override { return 0.9; }
   double getWeightFactor() const override { return 1.1; }
};

// Проверка входных данных
class PersonValidator {
public:
   void validate(const PersonData& p) const {
      if (p.height < 50 || p.height > 250) throw std::invalid_argument("Рост должен быть от 50 до 250 см");
      if (p.weight < 20 || p.weight > 300) throw std::invalid_argument("Вес должен быть от 20 до 300 кг");
      if (p.age < 14 || p.age > 100) throw std::invalid_argument("Возраст должен быть от 14 до 100 лет");
   }
};

// Основной обмен по формуле Миффлина - Сан Жеора, ккал
class BasalMetabolism {
public:
```

```
    double calculate(const PersonData& p) const {
        double base = 10 * p.weight + 6.25 * p.height - 5 * p.age;
        return p.sex == Sex::Male ? base + 5 : base - 161;
    }
};

// Коэффициент физической активности
class ActivityFactor {
public:
    double getFactor(Activity a) const {
        switch (a) {
        case Activity::Low: return 1.2;
        case Activity::Medium: return 1.55;
        default: return 1.725;
        }
    }

    std::string getName(Activity a) const {
        switch (a) {
        case Activity::Low: return "низкая";
        case Activity::Medium: return "средняя";
        default: return "высокая";
        }
    }
};

// Идеальный вес по формуле Брока с поправкой на телосложение, кг
class IdealWeight {
public:
    double calculate(double height, const BodyType& body) const {
        double base;
        if (height < 165) base = height - 100;
        else if (height <= 175) base = height - 105;
        else base = height - 110;
        return base * body.getWeightFactor();
    }
};

// ====================== Фасад ======================

// Единая точка входа: клиент передает данные человека и тип телосложения,
// а фасад сам обращается к классам подсистемы.
class CalorieFacade {
public:
    CalorieFacade() {
        bodyTypes[BodyKind::Asthenic] = std::unique_ptr<BodyType>(new Asthenic());
```

```
        bodyTypes[BodyKind::Normosthenic] = std::unique_ptr<BodyType>(new Normosthenic());
        bodyTypes[BodyKind::Hypersthenic] = std::unique_ptr<BodyType>(new Hypersthenic());
    }

    // Суточная норма, ккал (округляется до целого)
    double dailyNorm(const PersonData& p, BodyKind kind) const {
        validator.validate(p);
        double norm = metabolism.calculate(p) * activity.getFactor(p.activity) *
                bodyTypes.at(kind)->getCalorieFactor();
        return std::round(norm);
    }

    double idealWeight(const PersonData& p, BodyKind kind) const {
        validator.validate(p);
        return idealWeightCalc.calculate(p.height, *bodyTypes.at(kind));
    }

    std::string report(const PersonData& p, BodyKind kind) const {
        double norm = dailyNorm(p, kind);
        double ideal = idealWeight(p, kind);
        std::ostringstream out;
        out << (p.sex == Sex::Male ? "Мужчина" : "Женщина") << ", " << p.age << " лет, "
            << p.height << " см, " << p.weight << " кг, "
            << bodyTypes.at(kind)->getName() << ", активность " << activity.getName(p.activity) << "\n"
            << "  Основной обмен: " << std::round(metabolism.calculate(p)) << " ккал\n"
            << "  Суточная норма: " << norm << " ккал\n"
            << "  Идеальный вес: " << ideal << " кг\n"
            << "  Рекомендация: " << recommendation(p.weight, ideal, norm);
        return out.str();
    }

private:
    // Ссылки на классы подсистемы
    PersonValidator validator;
    BasalMetabolism metabolism;
    ActivityFactor activity;
    IdealWeight idealWeightCalc;
    std::map<BodyKind, std::unique_ptr<BodyType>> bodyTypes;

    std::string recommendation(double weight, double ideal, double norm) const {
        std::ostringstream out;
        if (weight > ideal * 1.1) {
            out << "для снижения веса около " << std::round(norm * 0.85) << " ккал";
        } else if (weight < ideal * 0.9) {
            out << "для набора веса около " << std::round(norm * 1.15) << " ккал";
        } else {
```

```
            out << "вес в норме, придерживайтесь " << norm << " ккал";
        }
        return out.str();
    }
};
}
```

4_GasAdapter.h

```
#pragma once

#include <string>
#include <sstream>
#include <stdexcept>

namespace Gas {

// Молярная газовая постоянная, Дж/(моль*К)
const double R = 8.31;

// ======================== Адаптируемый класс ========================

// Баллон с газом. Атрибуты оформлены как свойства (get/set),
// аналог автоматических свойств C#.
// Уравнение Менделеева-Клапейрона: pV = (m / M) * R * T
class GasCylinder {
public:
    GasCylinder(double volume, double mass, double molar) {
        setVolume(volume);
        setMass(mass);
        setMolar(molar);
    }

    // Объём баллона, м3
    double getVolume() const { return volume; }
    void setVolume(double value) {
        if (value <= 0) throw std::invalid_argument("Объём должен быть больше нуля");
        volume = value;
    }

    // Масса газа, кг
    double getMass() const { return mass; }
    void setMass(double value) {
        if (value < 0) throw std::invalid_argument("Масса не может быть отрицательной");
        mass = value;
    }

    // Молярная масса газа, кг/моль
    double getMolar() const { return molar; }
```

```
    void setMolar(double value) {
        if (value <= 0) throw std::invalid_argument("Молярная масса должна быть больше нуля");
        molar = value;
    }

    // Давление в баллоне при температуре T (К), Па
    double getPressure(int T) const {
        return amountOfMatter() * R * T / volume;
    }

    // Количество вещества, моль
    double amountOfMatter() const {
        return mass / molar;
    }

    std::string toString() const {
        std::ostringstream out;
        out << "Объём = " << volume << " м3, масса = " << mass
            << " кг, молярная масса = " << molar << " кг/моль";
        return out.str();
    }

private:
    double volume;
    double mass;
    double molar;
};

// ======================== Целевой интерфейс ========================

// Интерфейс, который ожидает клиент
class ICylinder {
public:
    virtual ~ICylinder() {}
    // Изменение давления при начальной температуре T0 и изменении температуры dT
    virtual double calculateDp(int T0, int dT) const = 0;
    // Изменить массу газа на величину dm
    virtual void modifMass(double dm) = 0;
    // Строка с данными об объекте
    virtual std::string getData() const = 0;
};

// ======================== Адаптер объектов ========================

// Адаптер хранит ссылку на адаптируемый объект (композиция)
// и переводит вызовы целевого интерфейса в вызовы GasCylinder.
class CylinderAdapter : public ICylinder {
public:
```

```
    CylinderAdapter(GasCylinder& cylinder) : cylinder(cylinder) {}

    double calculateDp(int T0, int dT) const override {
        return cylinder.getPressure(T0 + dT) - cylinder.getPressure(T0);
    }

    void modifMass(double dm) override {
        cylinder.setMass(cylinder.getMass() + dm);
    }

    std::string getData() const override {
        std::ostringstream out;
        out << "Баллон с газом: " << cylinder.toString()
            << ", количество вещества = " << cylinder.amountOfMatter() << " моль";
        return out.str();
    }

private:
    GasCylinder& cylinder;
};
}
```

4_Practice.cpp

```
#include <iostream>
#include <string>
#include <cmath>
#include <clocale>
#include "4_GasAdapter.h"
#include "4_CalorieFacade.h"

using namespace std;
using namespace Gas;
using namespace Calories;

// ======================== Клиент адаптера ========================

// Клиент знает только целевой интерфейс ICylinder
void useCylinder(ICylinder& cylinder) {
    cout << cylinder.getData() << endl;
    cout << "Изменение давления при нагреве с 300 К на 50 К: "
         << cylinder.calculateDp(300, 50) << " Па" << endl;
    cylinder.modifMass(0.5);
    cout << "После добавления 0.5 кг газа:" << endl;
    cout << cylinder.getData() << endl;
    cout << "Изменение давления при нагреве с 300 К на 50 К: "
         << cylinder.calculateDp(300, 50) << " Па" << endl;
}
```

```
// ======================== Тестирование ========================

class Tester {
public:
   Tester() : passed(0), failed(0) {}

   void check(bool condition, const string& name) {
      if (condition) {
         passed++;
         cout << "  [OK]   " << name << endl;
      } else {
         failed++;
         cout << "  [FAIL] " << name << endl;
      }
   }

   void printResult() const {
      cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
   }

private:
   int passed;
   int failed;
};

bool equal(double a, double b) { return fabs(a - b) < 1e-6; }

bool contains(const string& text, const string& part) {
   return text.find(part) != string::npos;
}

void testCylinder(Tester& t) {
   // 1 кг кислорода в баллоне 0.05 м3
   GasCylinder cylinder(0.05, 1, 0.032);
   t.check(equal(cylinder.amountOfMatter(), 31.25), "Количество вещества: 1 / 0.032 = 31.25 моль");
   t.check(equal(cylinder.getPressure(300), 1558125), "Давление при 300 К = 1558125 Па");

   bool thrown = false;
   try { GasCylinder bad(0, 1, 0.032); } catch (const invalid_argument&) { thrown = true; }
   t.check(thrown, "Баллон с нулевым объёмом не создается");
}

void testAdapter(Tester& t) {
   GasCylinder cylinder(0.05, 1, 0.032);
   CylinderAdapter adapter(cylinder);
   ICylinder& target = adapter;

   t.check(equal(target.calculateDp(300, 10), 51937.5), "calculateDp(300, 10) = 51937.5 Па");
```

```
    t.check(equal(target.calculateDp(300, 0), 0), "Без изменения температуры давление не меняется");
    t.check(target.calculateDp(300, -10) < 0, "При охлаждении давление падает");

    target.modifMass(1);
    t.check(equal(cylinder.getMass(), 2), "modifMass(+1) меняет массу адаптируемого баллона");
    target.modifMass(-0.5);
    t.check(equal(cylinder.getMass(), 1.5), "modifMass(-0.5) уменьшает массу");

    bool thrown = false;
    try { target.modifMass(-10); } catch (const invalid_argument&) { thrown = true; }
    t.check(thrown && equal(cylinder.getMass(), 1.5), "Нельзя сделать массу отрицательной");

    t.check(contains(target.getData(), "Баллон с газом") && contains(target.getData(), "моль"),
            "getData() возвращает данные баллона");
}

void testFacade(Tester& t) {
    CalorieFacade facade;
    PersonData man = { 180, 80, 30, Sex::Male, Activity::Low };
    PersonData woman = { 180, 80, 30, Sex::Female, Activity::Low };

    // Основной обмен мужчины: 10*80 + 6.25*180 - 5*30 + 5 = 1780
    t.check(equal(facade.dailyNorm(man, BodyKind::Normosthenic), 2136), "Норма нормостеника: 1780 * 1.2 =
2136");
    // Женщины: 1780 - 166 = 1614, 1614 * 1.2 = 1936.8
    t.check(equal(facade.dailyNorm(woman, BodyKind::Normosthenic), 1937), "Норма женщины меньше: 1937");

    t.check(facade.dailyNorm(man, BodyKind::Asthenic) > facade.dailyNorm(man, BodyKind::Normosthenic) &&
            facade.dailyNorm(man, BodyKind::Normosthenic) > facade.dailyNorm(man, BodyKind::Hypersthenic),
            "Астенику нужно больше калорий, гиперстенику меньше");

    PersonData active = man;
    active.activity = Activity::High;
    t.check(equal(facade.dailyNorm(active, BodyKind::Normosthenic), 3071), "Высокая активность: 1780 * 1.725 =
3071");

    t.check(equal(facade.idealWeight(man, BodyKind::Normosthenic), 70), "Идеальный вес при росте 180: 70 кг");
    t.check(equal(facade.idealWeight(man, BodyKind::Asthenic), 63), "Идеальный вес астеника: 63 кг");

    PersonData wrong = { 180, 80, 5, Sex::Male, Activity::Low };
    bool thrown = false;
    try { facade.dailyNorm(wrong, BodyKind::Normosthenic); } catch (const invalid_argument&) { thrown = true; }
    t.check(thrown, "Неверные данные отклоняются");

    t.check(contains(facade.report(man, BodyKind::Hypersthenic), "гиперстеник"), "Отчет фасада");
}

void runTests() {
```

```
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testCylinder(t);
    testAdapter(t);
    testFacade(t);
    t.printResult();
}

// ====================== Демонстрация ======================

int main() {
    setlocale(LC_ALL, "Russian");

    cout << "===== Паттерн Адаптер =====" << endl;
    GasCylinder oxygen(0.05, 1, 0.032);   // кислород, M = 32 г/моль
    CylinderAdapter adapter(oxygen);
    useCylinder(adapter);

    cout << "\n===== Паттерн Фасад =====" << endl;
    CalorieFacade facade;
    PersonData anna = { 165, 52, 20, Sex::Female, Activity::Medium };
    PersonData ivan = { 182, 95, 35, Sex::Male, Activity::Low };
    PersonData oleg = { 175, 68, 25, Sex::Male, Activity::High };

    cout << facade.report(anna, BodyKind::Asthenic) << endl;
    cout << facade.report(ivan, BodyKind::Hypersthenic) << endl;
    cout << facade.report(oleg, BodyKind::Normosthenic) << endl;

    try {
        PersonData wrong = { 300, 70, 20, Sex::Male, Activity::Low };
        facade.dailyNorm(wrong, BodyKind::Normosthenic);
    } catch (const invalid_argument& e) {
        cout << "Ошибка: " << e.what() << endl;
    }

    runTests();
    return 0;
}
```